



Московский государственный университет имени
М.В. Ломоносова

Факультет вычислительной математики и кибернетики
Кафедра алгоритмических языков

Огай Владислав Леонидович
Исмурзенов Абай Ерденович

Шахматы

Практическое задание

Преподаватели:

Шамаева Елена Денисовна
Елисеева Мария Алексеевна
Уразов Станислав Олегович

Москва, 2026

Содержание

1	Постановка задачи	3
2	Теоретическая часть	3
3	Описание программы	4
3.1	Модули	4
3.2	Структуры данных и состояние игры	5
3.3	Генерация и проверка ходов	5
3.4	Определение шаха, мата и ничьей	6
3.5	Логика бота	6
3.6	Цикл приложения и взаимодействие с пользователем	6
4	Используемые библиотеки	7
5	Пример работы программы	7
5.1	Основной интерфейс	7
5.2	Работа программы	10
5.3	Обработка ошибочных ситуаций	12
6	Отзывы	12
6.1	Отзыв: Андреев Илья Андреевич 324 АЯ	12
6.2	Отзыв: Соловьев Ярослав Сергеевич 324 АЯ	12
6.3	Отзыв: Ивановский Данила Сергеевич 324 АЯ	12
6.4	Отзыв: Коробов Евгений Алексеевич 324 АЯ	12
6.5	Отзыв: Морозов Кирилл Олегович 317 ММП	13
6.6	Отзыв: Григорьев Александр Сергеевич 219	13
6.7	Отзыв: Капитонов Станислав Сергеевич 219	13
7	Использование нейросетевых средств	13
8	Выводы	14
9	Литература	15

1 Постановка задачи

Цель работы заключается в создании компьютерной игры Шахматы на языке программирования Haskell. Требуется реализовать классические правила игры, графический интерфейс пользователя и бота для одиночной игры. Программа должна позволять делать ходы, брать фигуры противника, отменять свои действия и начинать партию заново по желанию пользователя.

2 Теоретическая часть

В основе реализации программы лежат классические правила игры в шахматы. Игра проходит на квадратной доске размером 8×8 клеток. У каждого игрока в начале игры имеется 16 фигур: один король, один ферзь, две ладьи, два слона, два коня и восемь пешек. Цель игры — поставить мат королю противника.

Каждая фигура перемещается по своим уникальным правилам:

- **Король** ходит на одну клетку в любом направлении.
- **Ферзь** перемещается на любое число пустых клеток по вертикали, горизонтали или диагонали.
- **Ладья** ходит на любое число пустых клеток по вертикали или горизонтали.
- **Слон** перемещается на любое число пустых клеток по диагонали.
- **Конь** ходит буквой «Г» (на две клетки в одном вертикальном или горизонтальном направлении и на одну в перпендикулярном), при этом он может перепрыгивать через другие фигуры.
- **Пешка** двигается на одну клетку вперед (со своей начальной позиции возможен ход на две клетки вперед), а бьет фигуры противника по диагонали на одну клетку вперед.

Кроме того, в программе учтены специальные правила:

- **Рокировка** — особый ход, в котором участвуют король и ладья. Разрешен, если обе фигуры еще не двигались, между ними нет других фигур, а король не находится под шахом и не пересекает битые поля.
- **Взятие на проходе** — взятие пешки противника, если та только что сделала ход на две клетки вперед и оказалась на соседней горизонтальной клетке.
- **Превращение пешки** — если пешка доходит до противоположного края доски, она заменяется на ферзя, ладью, слона или коня своего цвета.

Шах объявляется, когда король находится под ударом вражеской фигуры. Если от шаха нет защиты, фиксируется мат, и игра заканчивается победой атакующего. Если игрок в свою очередь не может сделать ни одного легального хода, но его король не под шахом, объявляется пат (ничья).

Для создания бота применяется алгоритм минимакс с альфа-бета отсечением. Алгоритм минимакс строит дерево возможных позиций на несколько ходов вперед: на четных уровнях бот выбирает ход, максимизирующий оценку позиции, а на нечетных — ход, минимизирующий её (предполагая наилучшие ответы противника). Альфа-бета отсечение ускоряет перебор, отбрасывая ветви алгоритма, которые уже не могут улучшить найденный результат.

Оценка позиции выполняется с помощью таблиц ценности фигур. Каждому типу фигуры соответствует матрица, задающая бонусные баллы за расположение на определенной клетке. Это позволяет боту оценивать не только материальное преимущество, но и позиционное: контроль центра, активное развитие фигур и безопасность короля.

3 Описание программы

3.1 Модули

Проект состоит из следующих файлов:

- `Types.hs` хранит базовые типы данных: цвет, координаты, различные фигуры, доску и общее состояние игры.
- `Board.hs` содержит функции для работы с клетками доски и для начальной расстановки всех фигур.
- `Moves.hs` отвечает за генерацию возможных путей перемещения для каждого типа фигур.
- `Rules.hs` проверяет правила. Здесь описывается проверка наличия шаха, мата и ничьей. Также описаны функции совершения хода на доске и его отмены.
- `Bot.hs` содержит алгоритм для выбора ходов компьютера на основе алгоритма минимакс и правила оценки шахматных позиций.
- `Events.hs` перехватывает клики мыши и нажатия кнопок на клавиатуре компьютера.

- `Render.hs` собирает текущую игровую сцену и рисует ее на экране: доску, меню, кнопки и текст с историей движений.
- `Lib.hs` связывает все компоненты вместе и задает бесконечный цикл обновления кадров приложения.
- `Main.hs` используется для запуска игры.

3.2 Структуры данных и состояние игры

Вся логика построена вокруг явного состояния игры, которое обновляется чистыми функциями. Состояние включает:

- доску 8×8 (в памяти как один массив из 64 элементов), где в каждой клетке либо пусто, либо лежит фигура с цветом;
- информацию о том, чей сейчас ход, выбранную игроком фигуру (если она подсвечена), а также историю сделанных ходов;
- режим партии (против человека или против бота), цвет игрока, а также флаги завершения партии (победа/ничья).

Любое действие (клик по фигуре, клик по клетке, кнопка отмены, перезапуск) является преобразованием `GameState -> GameState`.

3.3 Генерация и проверка ходов

Программа разделяет две задачи: *геометрически допустимые перемещения* и *ходы, разрешенные правилами*. Сначала для выбранной фигуры строится список кандидатов по правилам ее хода:

- для ладьи и слона генерируются лучи по направлениям до первой занятой клетки;
- для ферзя объединяются правила ладьи и слона;
- для коня используются фиксированные смещения на $(\pm 1, \pm 2)$ и $(\pm 2, \pm 1)$;
- для короля проверяются соседние клетки;
- для пешки обрабатываются шаг вперед, двойной шаг со стартовой линии и взятие по диагонали.

Далее каждый кандидат фильтруется правилами: нельзя брать свою фигуру, нельзя выходить за доску, а также запрещены ходы, после которых собственный король остается под атакой. Для этого ход временно применяется к доске, и выполняется

проверка состояния короля под шахом для полученной позиции. Такой подход гарантирует, что подсветка возможных клеток совпадает с тем, что реально разрешено сделать.

3.4 Определение шаха, мата и ничьей

Проверка шаха сводится к поиску атак на клетку короля со стороны фигур противника. После каждого хода программа:

- находит позицию короля текущего игрока;
- определяет, существует ли хотя бы одна атакующая фигура (по направлениям для дальнобойных фигур и по шаблонам для коня/пешек/короля).

Мат определяется как ситуация, когда король под шахом и при этом у игрока нет ни одного легального хода, который устраняет шах. Аналогично, ничья (пат) определяется как отсутствие легальных ходов при отсутствии шаха. В обоих случаях игра помечается завершенной, а интерфейс отображает соответствующее сообщение.

3.5 Логика бота

Обработка ходов компьютерного оппонента реализована в модуле `Bot.hs`. При запросе ответа программа генерирует список всех легальных перемещений на заданную глубину вычислений. Каждая финальная доска передается в оценочную функцию, чтобы определить числовую полезность данной ветки дерева ходов.

Архитектура бота спроектирована так, чтобы минимизировать накладные расходы на копирование структур при поиске. Если на каком-то этапе дальнейший перебор ответных ходов признается нерациональным, генерация этой ветки прерывается. По завершении анализа модуль возвращает выбранный ход в основной цикл приложения для преобразования текущего состояния `GameState`. В программе реализован базовый уровень сложности, была выбрана глубина 3, которая позволяет боту играть разумно, но не слишком сложно для начинающих игроков.

3.6 Цикл приложения и взаимодействие с пользователем

Графический интерфейс реализован с помощью `gloss`. Главные элементы: отрисовка текущей доски, подсветка выбранной фигуры и клеток для хода, а также кнопки сдать, перезапуск и отмена хода. Пользовательские действия обрабатываются в модуле `Events.hs`:

- клик по своей фигуре переводит ее в состояние выбранной и вычисляет список легальных ходов;
- клик по целевой клетке применяет ход, обновляет историю и проверяет завершение партии;
- в режиме игры с ботом после хода человека запускается выбор ответа бота и затем применяется ход компьютера.

Отмена хода реализована через хранение истории: достаточно восстановить предыдущее состояние доски и служебных полей и заново пересчитать подсветку/статус партии.

4 Используемые библиотеки

Для написания программы использовались следующие библиотеки:

- gloss применяется для создания графического окна и отрисовки плоской графики. Библиотека позволяет легко отслеживать клики и рисовать элементы из картинок.
- vector применяется для быстрого хранения состояния. Игровое поле в памяти компьютера представляет собой один сплошной массив из шестидесяти четырех элементов.

5 Пример работы программы

5.1 Основной интерфейс

При запуске открывается окно с главным меню. Человек может выбрать нужный вариант: партия против друга или против компьютера. Если выбран компьютер, система предлагает выбрать желаемый цвет своих стартовых фигур. На основном экране показана большая доска, кнопки признания поражения и быстрого перезапуска, а также отмена хода. Вверху выводится текстовая история матча.



Рис. 1: Главное меню программы

Меню позволяет выбрать режим игры и запустить новую партию. В этом же месте выполняется выбор параметров одиночной игры, после чего программа переходит на экран доски и начинает партию с начальной расстановки фигур.



Рис. 2: Пример игрового процесса: разыграно несколько ходов

На данном примере (рис. 2) видно, как происходит выбор фигуры и подсветка допустимых клеток. После клика по своей фигуре программа строит список легальных ходов и показывает их на доске, поэтому игрок сразу понимает, какие перемещения разрешены правилами. После выполнения хода позиция на доске и строка истории обновляются синхронно, что помогает отслеживать ход партии.

5.2 Работа программы

Сначала пользователь кликает левой кнопкой мыши на свою фигуру, при этом клетка под ней загорается зеленым цветом. Клетки, на которые можно сделать шаг, отмечаются серыми кругами. При клике на нужную пустую клетку фигура мгновенно перемещается туда. Если включена игра против бота, компьютер делает свой ход, при этом время ожидания зависит от глубины поиска и вычислительной мощности компьютера. Завершенные шаги сразу отображаются в текстовой строке сверху.



Рис. 3: Ситуация шах: король находится под атакой

Когда возникает шах, программа вычисляет, атакована ли клетка короля фигурами противника. В этом состоянии разрешены только те ходы, которые убирают шах: уход короля, взятие атакующей фигуры или блокировка линии атаки (если это возможно). Любые попытки выполнить ход, оставляющий короля под ударом, автоматически отклоняются, поэтому пользователь не может нарушить правила.

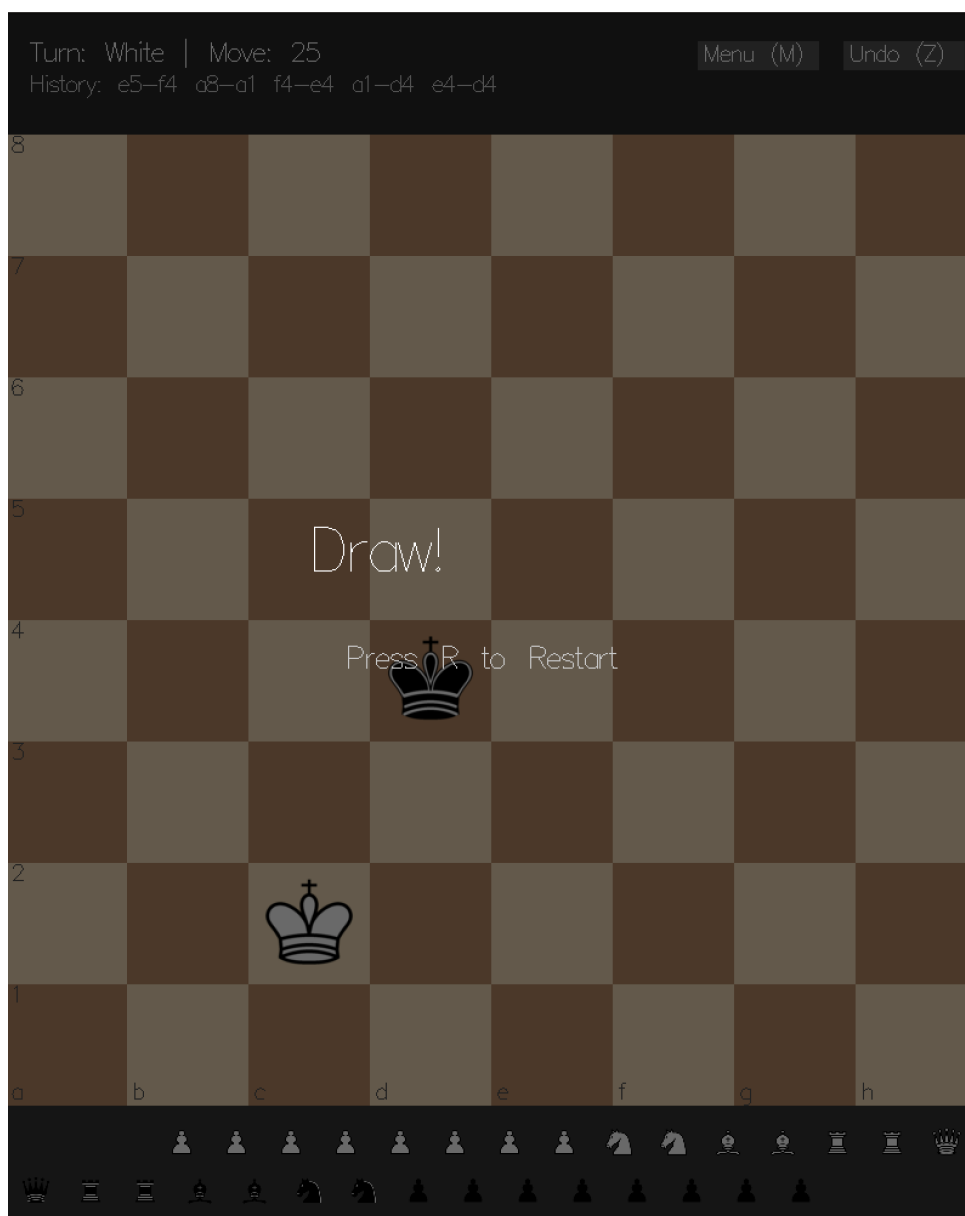


Рис. 4: Ситуация ничьей: у игрока нет легальных ходов (пат) или зафиксировано завершение партии вничью

В ситуации ничьей программа проверяет, что у текущего игрока нет ни одного легального хода. Если при этом король не находится под шахом, фиксируется

пат. После определения результата партия переводится в завершенное состояние, и дальнейшие ходы не выполняются, пока пользователь не начнет новую игру или не воспользуется перезапуском.

5.3 Обработка ошибочных ситуаций

Программа не позволяет человеку нарушить четкие правила шахмат. Если перемещение ставит собственного короля под прямой удар или король уже атакован, защита запретит двигать другие фигуры. Клик мышкой далеко за пределы кнопок или на пустую клетку просто игнорируется и никак не ломает игру.

6 ОТЗЫВЫ

6.1 Отзыв: Андреев Илья Андреевич 324 АЯ

Программа мне понравилась, авторы оставили указания с возможными действиями что очень помогло. Так же порадовал знакомый интерфейс и возможность игры против бота. Из предложений: ввести несколько уровней сложности бота Минусы: предыдущий ход ии не показывается, ии играет один и тот же дебют

6.2 Отзыв: Соловьев Ярослав Сергеевич 324 АЯ

Как шахматы вполне нормально. Интерфейс очень простой. Отсутствует возможность выбора сложности бота. При попытке управления через клавиатуру работает только с английской раскладкой.

6.3 Отзыв: Ивановский Данила Сергеевич 324 АЯ

Игра работает, интерфейс понятный. Бот играет часто однотипо. Было бы круто добавить возможность выбора сложности бота и отображение последнего хода.

6.4 Отзыв: Коробов Евгений Алексеевич 324 АЯ

Очень старался сломать программу. Но не получилось. Очень расстроился.

6.5 Отзыв: Морозов Кирилл Олегович 317 ММП

Игра работает корректно, багов из разряда несуществующих ходов или типа того не обнаружил. Визуально, игра тоже сделана довольно хорошо. Есть пару моментов: 1) Не на весь экран почему-то отображалась игра. У меня их не видно истории ходов и другие статистики игры. 2) Довольно часто компьютер после моего хода долго думает. Я снова думаю это связано с мощностями CPU (считаются сразу 5 следующих ходов, не самые лёгкие подсчёты, наверно). Больше 5 секунд ожидания. 3) Как раз к прошлому пункту: выбор сложности был бы не плох. 4) добавить рандомайзер ходов. В остальном, очень крутой проект, я считаю (тестил только PvE режим, если что).

6.6 Отзыв: Григорьев Александр Сергеевич 219

Бот в целом неплох, но в дебюте играет очень пассивно, при этом развивая ферзя в начале, что считается неверной стратегией в общем случае.

Также могу сказать, что бот теряется в эндшпиле из-за маленькой глубины. Так как на этом этапе игры фигур мало, можно попробовать поднять глубину без сильной потери производительности.

6.7 Отзыв: Капитонов Станислав Сергеевич 219

Бот неплох, дебют и миттельшпиль играет нормально, но в худшей позиции отказывается от трехкратного повторения, тем самым допустив сформированный мат.

7 Использование нейросетевых средств

При проектировании архитектуры и написании кода для данного проекта диалоговые модели искусственного интеллекта применялись преимущественно в качестве интерактивного справочника. В самом начале работы над бэкендом (логическим движком) для консультации по правильному дизайну приложения нейросети был задан следующий промпт:

«Я с товарищем собираемся написать шахматы на языке Хаскель. Это движок правил; типы данных: фигуры, цвет, клетка, доска, GameState; генерация псевдоходов для всех фигур; применение хода к состоянию; проверка шаха и фильтр

легальности; спецправила: рокировка, en passant, превращение. Как должен быть организован проект? Хороша ли идея разбивать его на файлы-модули? На какие?»

Опираясь на ответ ИИ как на структурный гайд, разработка была логично структурирована и поделена на смысловые файлы-модули (Board, Rules, Moves и др.), что сильно упростило параллельную работу соавторов. Выжимки из документации, предоставляемые нейросетью, помогли выбрать правильные абстракции для Haskell без необходимости долгого изучения тематических форумов.

Для реализации фронтенда нейросеть также выступала в роли справочника, но уже для уточнения особенностей функциональной библиотеки `gloss`. Задавались вопросы уточняющего характера: *«Моя часть с Gloss... подскажи, как работает библиотека, как лучше организовать логику отрисовки и событий?»*. Поскольку библиотека использует специфический декларативный подход к событийному циклу, искусственный интеллект помог разобраться с тем, как транслировать чистые функции Haskell в непрерывное обновление окна. Он также подсказал удобные математические формулы для перевода курсора мыши (экранных пикселей) в индексы шахматной клетки. Это сработало как отличная выдержка из официальной документации и позволило быстро связать графический интерфейс с логическим движком.

Кроме того, нейросетевые средства помогли решить проблему дистрибуции игры. Был задан вопрос: *«Можно ли собрать приложение в работающий .exe файл так, чтобы людям не нужно было скачивать Stack/cabal? Что такое .github/workflows/release.yml и как он должен выглядеть?»*. ИИ объяснил механику компиляции Haskell в независимый машинный код и предоставил готовый шаблон конфигурации для GitHub Actions. Благодаря этому теперь существуют готовые исполняемые программы (релизы) для Windows, macOS и Linux.

8 Выводы

В результате работы создана полная игра Шахматы. Реализован логичный графический интерфейс, проверка шахматных правил и умный компьютерный противник для одиночных партий. Проект показал пользу и удобство функционального подхода языка Haskell для реализации шахматной логики. Для проекта было собрано три релиза для разных операционных систем. С проектом можно ознакомиться по ссылке: <https://github.com/ovladisluvv/Haskell-Chess-2026>

9 Литература

1. Hackage. Gloss: библиотека для простой 2D-графики и обработки событий. URL: <https://hackage.haskell.org/package/gloss>.
2. The Haskell 2010 Report. Официальное описание языка Haskell. URL: <https://www.haskell.org/onlinereport/haskell2010/>.
3. Chessprogramming Wiki. 8x8 Board: представление шахматной доски в виде массива. URL: https://www.chessprogramming.org/8x8_Board.
4. Chessprogramming Wiki. Vector Attacks: вычисление атак дальнобойных фигур по направлениям. URL: https://www.chessprogramming.org/Vector_Attacks.
5. Chessprogramming Wiki. Minimax: алгоритм поиска лучшего хода в играх. URL: <https://www.chessprogramming.org/Minimax>.
6. Chessprogramming Wiki. Alpha-Beta: отсечение ветвей в минимаксе. URL: <https://www.chessprogramming.org/Alpha-Beta>.
7. Chessprogramming Wiki. Simplified Evaluation Function: пример простой оценочной функции позиции. URL: https://www.chessprogramming.org/Simplified_Evaluation_Function.
8. Chessprogramming Wiki. Mobility: учёт подвижности фигур в оценке позиции. URL: <https://www.chessprogramming.org/Mobility>.